

```
In [34]: # Author: Krutarth Parmar
# Date: 2025-04-03
```

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
import torch.nn as nn
import torch.nn.functional as F
import torch
import tensorflow as tf
import tensorflow.keras
```

VAE Model instansiation

```
In [3]: latent_dim = 16
intermediate_dim = 128

class VAE(nn.Module):
    def __init__(self, latent_dim=16, intermediate_dim=128):
        super(VAE, self).__init__()

        # --- Encoder ---
        self.conv1 = nn.Conv2d(1, 32, kernel_size=4, stride=2, padding=1) #
        self.bn1 = nn.BatchNorm2d(32) # Added BatchNorm

        self.conv2 = nn.Conv2d(32, 64, kernel_size=4, stride=2, padding=1) #
        self.bn2 = nn.BatchNorm2d(64) # Added BatchNorm
        # Flattened size: 64 * 7 * 7 = 3136
        self.fc1 = nn.Linear(64 * 7 * 7, intermediate_dim)
        self.bn3 = nn.BatchNorm1d(intermediate_dim) # Added BatchNorm

        self.fc_mu = nn.Linear(intermediate_dim, latent_dim)
        self.fc_logvar = nn.Linear(intermediate_dim, latent_dim)

        # --- Decoder ---
        self.fc_decode1 = nn.Linear(latent_dim, intermediate_dim)
        self.bn4 = nn.BatchNorm1d(intermediate_dim) # Added BatchNorm
        self.fc_decode2 = nn.Linear(intermediate_dim, 64 * 7 * 7)
        self.bn5 = nn.BatchNorm1d(64 * 7 * 7) # Added BatchNorm

        self.conv_t1 = nn.ConvTranspose2d(64, 32, kernel_size=4, stride=2, padding=1) #
        self.bn6 = nn.BatchNorm2d(32) # Added BatchNorm
        self.conv_t2 = nn.ConvTranspose2d(32, 1, kernel_size=4, stride=2, padding=1) #
        # NO BatchNorm before final Sigmoid

    def encode(self, x):
        # Apply Conv -> BatchNorm -> ReLU
        x = F.relu(self.bn1(self.conv1(x)))
        x = F.relu(self.bn2(self.conv2(x)))
        x = x.view(x.size(0), -1) # Flatten
        x = F.relu(self.bn3(self.fc1(x)))
        mu = self.fc_mu(x)
        logvar = self.fc_logvar(x) # No activation/BN on mu/logvar outputs
```

```

        return mu, logvar

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def decode(self, z):
        # Apply Linear -> BatchNorm -> ReLU
        z = F.relu(self.bn4(self.fc_decode1(z)))
        z = F.relu(self.bn5(self.fc_decode2(z)))
        z = z.view(z.size(0), 64, 7, 7) # Reshape
        # Apply ConvTranspose -> BatchNorm -> ReLU
        z = F.relu(self.bn6(self.conv_t1(z)))
        # Final layer -> Sigmoid (NO BatchNorm here)
        reconstruction = torch.sigmoid(self.conv_t2(z))
        return reconstruction

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        recon_x = self.decode(z)
        return recon_x, mu, logvar

```

```

In [4]: model = VAE(latent_dim=latent_dim, intermediate_dim=intermediate_dim)
        model.load_state_dict(torch.load('vae_rotated_mnist_1_2_epoch8.pth'))

```

/var/folders/j3/l5m7q9lx3gqff5xrl7n7sck40000gn/T/ipykernel_23228/1788141676.py:2: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowed by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

```

        model.load_state_dict(torch.load('vae_rotated_mnist_1_2_epoch8.pth'))

```

```

Out[4]: <All keys matched successfully>

```

Load MNIST Rotated Data for digits 1 & 2

```

In [5]: from scipy.ndimage import rotate

        # download mnist data
        (x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
        x_digits = []
        y_labels = [] # To store 1 or 2

        for i in range(len(x_train)):

```

```

    if y_train[i] == 1 or y_train[i] == 2:
        x_digits.append(x_train[i])
        y_labels.append(y_train[i]) # Store the label (1 or 2)

print(f"Found {len(x_digits)} images of digits 1 and 2.")
# Convert labels to numpy array for easier handling later if needed
y_labels = np.array(y_labels)

```

Found 12700 images of digits 1 and 2.

```

In [17]: from scipy.ndimage import rotate
import numpy as np

rotated_images_all = {} # Dictionary to store images by angle
angles = range(0, 360, 30) # 0, 30, 60, ..., 330

for angle in angles:
    print(f"Rotating images by {angle} degrees...")
    rotated_list_for_angle = []
    if angle == 0:
        # No rotation needed for 0 degrees
        rotated_list_for_angle = [img for img in x_digits] # Use original in
    else:
        for img in x_digits:
            rotated_img = rotate(img, angle, reshape=False, mode='nearest')
            rotated_list_for_angle.append(rotated_img)

    # Store the list of numpy arrays for this angle
    rotated_images_all[angle] = np.array(rotated_list_for_angle) # Store as

print("Finished rotating images for all angles.")

```

```

Rotating images by 0 degrees...
Rotating images by 30 degrees...
Rotating images by 60 degrees...
Rotating images by 90 degrees...
Rotating images by 120 degrees...
Rotating images by 150 degrees...
Rotating images by 180 degrees...
Rotating images by 210 degrees...
Rotating images by 240 degrees...
Rotating images by 270 degrees...
Rotating images by 300 degrees...
Rotating images by 330 degrees...
Finished rotating images for all angles.

```

```

In [7]: import torch

model.eval() # Set model to evaluation mode

latent_vectors_all = {} # Dictionary to store latent vectors (mu) by angle

with torch.no_grad(): # Disable gradient calculations
    for angle in angles:
        print(f"Encoding images for {angle} degrees...")
        images_np = rotated_images_all[angle]

```

```

# Convert numpy array to PyTorch tensor, add channel dimension
images_tensor = torch.tensor(images_np, dtype=torch.float32).unsqueeze(1)
images_tensor = images_tensor / 255.0

mu, logvar = model.encode(images_tensor)

# Store the mu vectors (latent representations) as numpy arrays
latent_vectors_all[angle] = mu.cpu().numpy()

print("Finished encoding images for all angles.")

```

```

Encoding images for 0 degrees...
Encoding images for 30 degrees...
Encoding images for 60 degrees...
Encoding images for 90 degrees...
Encoding images for 120 degrees...
Encoding images for 150 degrees...
Encoding images for 180 degrees...
Encoding images for 210 degrees...
Encoding images for 240 degrees...
Encoding images for 270 degrees...
Encoding images for 300 degrees...
Encoding images for 330 degrees...
Finished encoding images for all angles.

```

```

In [8]: X_mlp = []
        y_mlp = []

        angles_list = list(angles) # [0, 30, ..., 330]

        for i in range(len(angles_list)):
            current_angle = angles_list[i]
            # Calculate the next angle, handling wrap-around (330 -> 0)
            next_angle_index = (i + 1) % len(angles_list)
            next_angle = angles_list[next_angle_index]

            print(f"Pairing: {current_angle}° -> {next_angle}°")

            # Get the latent vectors for the current and next angles
            z_theta = latent_vectors_all[current_angle] # Shape: (num_images, latent_dim)
            z_theta_plus_30 = latent_vectors_all[next_angle] # Shape: (num_images, latent_dim)

            # Add them to our MLP training lists
            X_mlp.append(z_theta)
            y_mlp.append(z_theta_plus_30)

        # Concatenate lists of arrays into single large arrays
        X_mlp = np.concatenate(X_mlp, axis=0) # Shape: (num_images * 12, latent_dim)
        y_mlp = np.concatenate(y_mlp, axis=0) # Shape: (num_images * 12, latent_dim)

        print(f"MLP Input Shape: {X_mlp.shape}")
        print(f"MLP Target Shape: {y_mlp.shape}")

```

```

Pairing: 0° -> 30°
Pairing: 30° -> 60°
Pairing: 60° -> 90°
Pairing: 90° -> 120°
Pairing: 120° -> 150°
Pairing: 150° -> 180°
Pairing: 180° -> 210°
Pairing: 210° -> 240°
Pairing: 240° -> 270°
Pairing: 270° -> 300°
Pairing: 300° -> 330°
Pairing: 330° -> 0°
MLP Input Shape: (152400, 16)
MLP Target Shape: (152400, 16)

```

```

In [9]: from sklearn.manifold import TSNE

# Determine the number of original images per angle block
# Assuming all angles have the same number of images/vectors
num_images_per_angle = latent_vectors_all[0].shape[0]
num_angles = len(angles_list)

# --- 1. Create Labels for Visualization (Original Angle) ---

angle_labels = []
for angle in angles_list:
    # Assign the current angle as the label for all images processed at this
    angle_labels.extend([angle] * num_images_per_angle)

angle_labels = np.array(angle_labels)

# Check if the length matches X_mlp's first dimension
if len(angle_labels) != X_mlp.shape[0]:
    print("Warning: Label count mismatch! Check data construction.")
    # Handle potential error or recalculate based on X_mlp.shape[0] if certa

print(f"Created {len(angle_labels)} labels for {X_mlp.shape[0]} data points.")

# --- 2. Apply t-SNE ---

print("Applying t-SNE... (This may take a while for large datasets)")
tsne = TSNE(n_components=2, random_state=42, perplexity=30, n_iter=300, init
X_mlp_2d = tsne.fit_transform(X_mlp)

print("t-SNE finished.")

# --- 3. Visualize the 2D Projection ---

print("Plotting t-SNE results...")
plt.figure(figsize=(12, 10))
scatter = plt.scatter(X_mlp_2d[:, 0], X_mlp_2d[:, 1], c=angle_labels, cmap='
plt.title('t-SNE Visualization of Latent Space (Colored by Rotation Angle)')
plt.xlabel('t-SNE Dimension 1')
plt.ylabel('t-SNE Dimension 2')

# Add a colorbar to show mapping from color to angle

```

```

cbar = plt.colorbar(scatter)
cbar.set_label('Rotation Angle (Degrees)')

plt.grid(True)
plt.show()
print("Plot displayed.")

```

Created 152400 labels for 152400 data points.

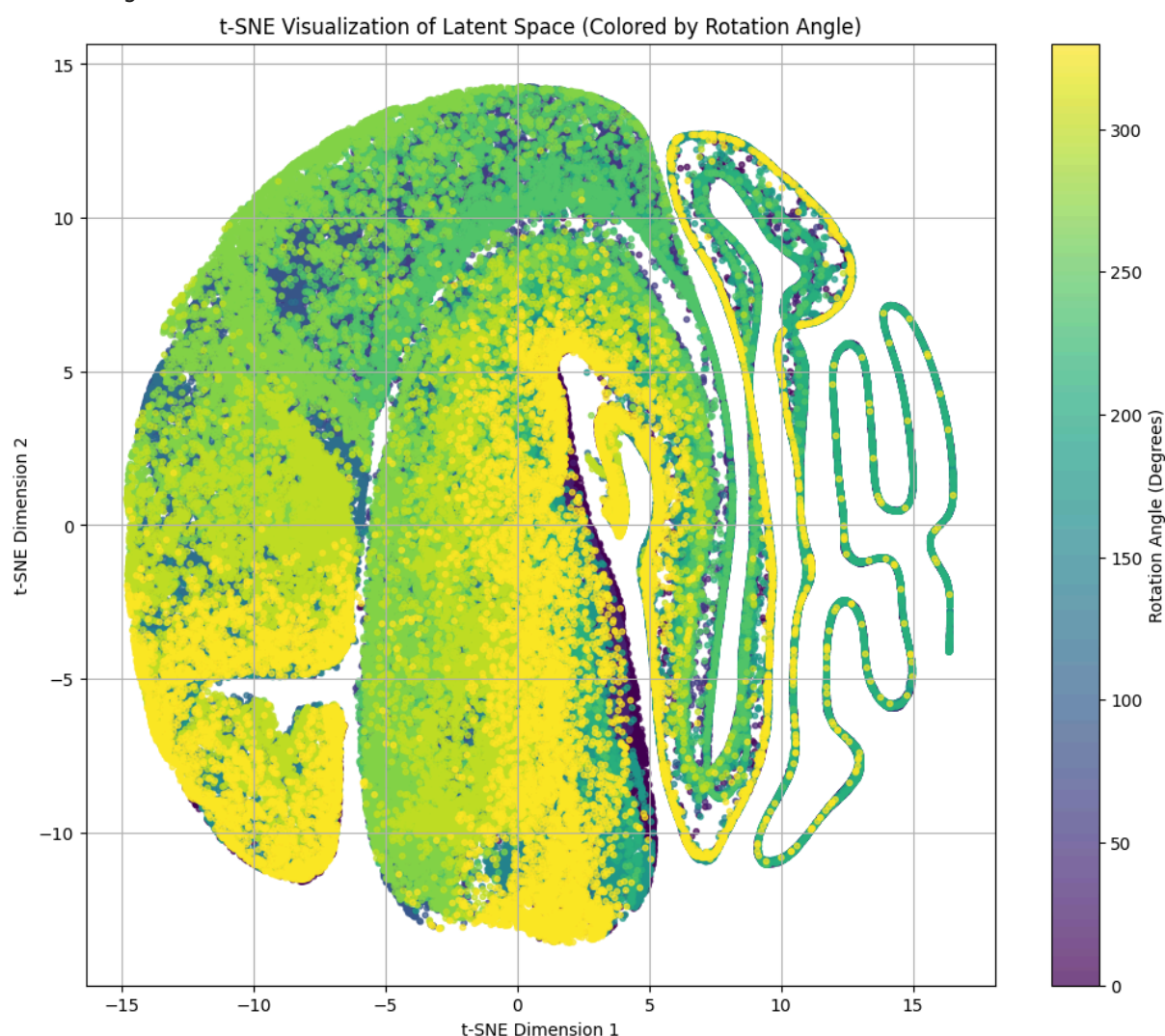
Applying t-SNE... (This may take a while for large datasets)

/opt/homebrew/anaconda3/envs/tf-m1/lib/python3.11/site-packages/sklearn/manifold/_t_sne.py:1164: FutureWarning: 'n_iter' was renamed to 'max_iter' in version 1.5 and will be removed in 1.7.

warnings.warn(

t-SNE finished.

Plotting t-SNE results...



Plot displayed.

```

In [10]: import torch.nn as nn

latent_dim = 16
hidden_dim = 128

mlp_model = nn.Sequential(
    nn.Linear(latent_dim, hidden_dim),

```

```

        nn.ReLU(),
        nn.Linear(hidden_dim, hidden_dim), # second hidden layer
        nn.ReLU(), # second activation
        nn.Linear(hidden_dim, latent_dim) # Final linear output layer
    )

    print(mlp_model)

```

```

Sequential(
  (0): Linear(in_features=16, out_features=128, bias=True)
  (1): ReLU()
  (2): Linear(in_features=128, out_features=128, bias=True)
  (3): ReLU()
  (4): Linear(in_features=128, out_features=16, bias=True)
)

```

```

In [11]: from torch.utils.data import TensorDataset, DataLoader
import torch.optim as optim

# --- 1. Prepare Data for PyTorch ---

# Convert NumPy arrays to PyTorch tensors
# Ensure dtype is torch.float32 as neural networks typically work with float
X_tensor = torch.tensor(X_mlp, dtype=torch.float32)
y_tensor = torch.tensor(y_mlp, dtype=torch.float32)

# Create a TensorDataset (combines inputs and targets)
dataset = TensorDataset(X_tensor, y_tensor)

# Create a DataLoader (handles batching and shuffling)
batch_size = 64 # Choose a suitable batch size
shuffle_data = True # Shuffle data each epoch
train_loader = DataLoader(dataset, batch_size=batch_size, shuffle=shuffle_data)

# --- 2. Define Loss Function and Optimizer ---

# Loss Function (Mean Squared Error for regression)
criterion = nn.MSELoss()

# Optimizer (Adam is a common choice)
learning_rate = 0.001 # Choose a suitable learning rate
optimizer = optim.Adam(mlp_model.parameters(), lr=learning_rate)

# --- 3. Training Loop ("Fitting") ---

# Choose device (GPU if available, otherwise CPU)
# device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

num_epochs = 20 # Choose the number of epochs

print("Starting training...")
for epoch in range(num_epochs):
    mlp_model.train() # Set the model to training mode
    running_loss = 0.0
    num_batches = 0

```

```
for batch_idx, (inputs, targets) in enumerate(train_loader):
    # Move data to the same device as the model
    # inputs, targets = inputs.to(device), targets.to(device)

    # --- Forward pass ---
    outputs = mlp_model(inputs)
    loss = criterion(outputs, targets)

    # --- Backward pass and optimization ---
    optimizer.zero_grad() # Clear previous gradients
    loss.backward()        # Calculate gradients
    optimizer.step()        # Update weights

    running_loss += loss.item()
    num_batches += 1

    # Print progress within an epoch
    if batch_idx % 100 == 99: # Print every 100 batches
        print(f'Epoch [{epoch+1}/{num_epochs}], Batch [{batch_idx+1}/{len(train_loader)}]')

    # Calculate average loss for the epoch
    epoch_loss = running_loss / num_batches
    print(f"Epoch [{epoch+1}/{num_epochs}] completed. Average Loss: {epoch_loss}")

print("Training finished.")

# --- 4. Save the trained model ---
torch.save(mlp_model.state_dict(), 'mlp_rotation_model_1_2.pth')
```

Starting training...

Epoch [1/20], Batch [100/2382], Loss: 516.0539
Epoch [1/20], Batch [200/2382], Loss: 1115.7460
Epoch [1/20], Batch [300/2382], Loss: 463.9506
Epoch [1/20], Batch [400/2382], Loss: 227.5825
Epoch [1/20], Batch [500/2382], Loss: 206.6644
Epoch [1/20], Batch [600/2382], Loss: 153.2731
Epoch [1/20], Batch [700/2382], Loss: 93.5533
Epoch [1/20], Batch [800/2382], Loss: 115.1141
Epoch [1/20], Batch [900/2382], Loss: 83.9413
Epoch [1/20], Batch [1000/2382], Loss: 158.5488
Epoch [1/20], Batch [1100/2382], Loss: 83.6100
Epoch [1/20], Batch [1200/2382], Loss: 70.2930
Epoch [1/20], Batch [1300/2382], Loss: 359.8156
Epoch [1/20], Batch [1400/2382], Loss: 129.7860
Epoch [1/20], Batch [1500/2382], Loss: 125.1040
Epoch [1/20], Batch [1600/2382], Loss: 82.6568
Epoch [1/20], Batch [1700/2382], Loss: 63.5876
Epoch [1/20], Batch [1800/2382], Loss: 52.2737
Epoch [1/20], Batch [1900/2382], Loss: 165.4908
Epoch [1/20], Batch [2000/2382], Loss: 55.0652
Epoch [1/20], Batch [2100/2382], Loss: 101.4938
Epoch [1/20], Batch [2200/2382], Loss: 52.9639
Epoch [1/20], Batch [2300/2382], Loss: 30.9382
Epoch [1/20] completed. Average Loss: 217.558325
Epoch [2/20], Batch [100/2382], Loss: 83.2320
Epoch [2/20], Batch [200/2382], Loss: 93.0129
Epoch [2/20], Batch [300/2382], Loss: 74.8169
Epoch [2/20], Batch [400/2382], Loss: 54.5460
Epoch [2/20], Batch [500/2382], Loss: 78.0418
Epoch [2/20], Batch [600/2382], Loss: 51.0844
Epoch [2/20], Batch [700/2382], Loss: 54.5203
Epoch [2/20], Batch [800/2382], Loss: 71.7356
Epoch [2/20], Batch [900/2382], Loss: 64.3922
Epoch [2/20], Batch [1000/2382], Loss: 56.7893
Epoch [2/20], Batch [1100/2382], Loss: 100.9071
Epoch [2/20], Batch [1200/2382], Loss: 101.8675
Epoch [2/20], Batch [1300/2382], Loss: 62.8502
Epoch [2/20], Batch [1400/2382], Loss: 63.8000
Epoch [2/20], Batch [1500/2382], Loss: 37.8770
Epoch [2/20], Batch [1600/2382], Loss: 53.6757
Epoch [2/20], Batch [1700/2382], Loss: 24.5968
Epoch [2/20], Batch [1800/2382], Loss: 37.9383
Epoch [2/20], Batch [1900/2382], Loss: 127.5167
Epoch [2/20], Batch [2000/2382], Loss: 34.1233
Epoch [2/20], Batch [2100/2382], Loss: 155.1531
Epoch [2/20], Batch [2200/2382], Loss: 49.4793
Epoch [2/20], Batch [2300/2382], Loss: 63.4283
Epoch [2/20] completed. Average Loss: 73.378010
Epoch [3/20], Batch [100/2382], Loss: 37.0891
Epoch [3/20], Batch [200/2382], Loss: 36.1291
Epoch [3/20], Batch [300/2382], Loss: 117.7033
Epoch [3/20], Batch [400/2382], Loss: 65.8294
Epoch [3/20], Batch [500/2382], Loss: 42.5051
Epoch [3/20], Batch [600/2382], Loss: 60.1920
Epoch [3/20], Batch [700/2382], Loss: 77.6672

Epoch [3/20], Batch [800/2382], Loss: 69.6093
Epoch [3/20], Batch [900/2382], Loss: 25.8718
Epoch [3/20], Batch [1000/2382], Loss: 121.4918
Epoch [3/20], Batch [1100/2382], Loss: 26.3229
Epoch [3/20], Batch [1200/2382], Loss: 52.2507
Epoch [3/20], Batch [1300/2382], Loss: 86.9195
Epoch [3/20], Batch [1400/2382], Loss: 259.7631
Epoch [3/20], Batch [1500/2382], Loss: 39.4257
Epoch [3/20], Batch [1600/2382], Loss: 63.7880
Epoch [3/20], Batch [1700/2382], Loss: 69.4538
Epoch [3/20], Batch [1800/2382], Loss: 61.1937
Epoch [3/20], Batch [1900/2382], Loss: 28.1646
Epoch [3/20], Batch [2000/2382], Loss: 57.4459
Epoch [3/20], Batch [2100/2382], Loss: 117.6334
Epoch [3/20], Batch [2200/2382], Loss: 41.5209
Epoch [3/20], Batch [2300/2382], Loss: 27.2824
Epoch [3/20] completed. Average Loss: 68.262831
Epoch [4/20], Batch [100/2382], Loss: 37.7465
Epoch [4/20], Batch [200/2382], Loss: 77.0369
Epoch [4/20], Batch [300/2382], Loss: 23.7327
Epoch [4/20], Batch [400/2382], Loss: 145.1117
Epoch [4/20], Batch [500/2382], Loss: 61.9935
Epoch [4/20], Batch [600/2382], Loss: 15.3926
Epoch [4/20], Batch [700/2382], Loss: 18.4160
Epoch [4/20], Batch [800/2382], Loss: 37.8019
Epoch [4/20], Batch [900/2382], Loss: 86.9382
Epoch [4/20], Batch [1000/2382], Loss: 50.0014
Epoch [4/20], Batch [1100/2382], Loss: 65.0336
Epoch [4/20], Batch [1200/2382], Loss: 69.2143
Epoch [4/20], Batch [1300/2382], Loss: 31.0056
Epoch [4/20], Batch [1400/2382], Loss: 24.8988
Epoch [4/20], Batch [1500/2382], Loss: 66.8965
Epoch [4/20], Batch [1600/2382], Loss: 93.9482
Epoch [4/20], Batch [1700/2382], Loss: 46.0629
Epoch [4/20], Batch [1800/2382], Loss: 18.4146
Epoch [4/20], Batch [1900/2382], Loss: 57.9539
Epoch [4/20], Batch [2000/2382], Loss: 30.7619
Epoch [4/20], Batch [2100/2382], Loss: 73.8321
Epoch [4/20], Batch [2200/2382], Loss: 42.2561
Epoch [4/20], Batch [2300/2382], Loss: 44.9030
Epoch [4/20] completed. Average Loss: 61.745700
Epoch [5/20], Batch [100/2382], Loss: 38.7830
Epoch [5/20], Batch [200/2382], Loss: 22.6645
Epoch [5/20], Batch [300/2382], Loss: 63.1520
Epoch [5/20], Batch [400/2382], Loss: 34.2821
Epoch [5/20], Batch [500/2382], Loss: 42.8877
Epoch [5/20], Batch [600/2382], Loss: 63.5338
Epoch [5/20], Batch [700/2382], Loss: 26.3758
Epoch [5/20], Batch [800/2382], Loss: 58.6908
Epoch [5/20], Batch [900/2382], Loss: 27.8284
Epoch [5/20], Batch [1000/2382], Loss: 86.2869
Epoch [5/20], Batch [1100/2382], Loss: 40.4851
Epoch [5/20], Batch [1200/2382], Loss: 22.9142
Epoch [5/20], Batch [1300/2382], Loss: 58.8200
Epoch [5/20], Batch [1400/2382], Loss: 21.6708
Epoch [5/20], Batch [1500/2382], Loss: 25.1696

Epoch [5/20], Batch [1600/2382], Loss: 88.2526
Epoch [5/20], Batch [1700/2382], Loss: 32.9908
Epoch [5/20], Batch [1800/2382], Loss: 47.3560
Epoch [5/20], Batch [1900/2382], Loss: 18.3077
Epoch [5/20], Batch [2000/2382], Loss: 75.6631
Epoch [5/20], Batch [2100/2382], Loss: 35.5710
Epoch [5/20], Batch [2200/2382], Loss: 184.5773
Epoch [5/20], Batch [2300/2382], Loss: 34.3694
Epoch [5/20] completed. Average Loss: 50.792305
Epoch [6/20], Batch [100/2382], Loss: 56.1704
Epoch [6/20], Batch [200/2382], Loss: 16.5534
Epoch [6/20], Batch [300/2382], Loss: 33.6719
Epoch [6/20], Batch [400/2382], Loss: 29.4389
Epoch [6/20], Batch [500/2382], Loss: 20.0138
Epoch [6/20], Batch [600/2382], Loss: 50.4693
Epoch [6/20], Batch [700/2382], Loss: 55.4664
Epoch [6/20], Batch [800/2382], Loss: 37.7114
Epoch [6/20], Batch [900/2382], Loss: 48.8907
Epoch [6/20], Batch [1000/2382], Loss: 27.3892
Epoch [6/20], Batch [1100/2382], Loss: 51.1039
Epoch [6/20], Batch [1200/2382], Loss: 44.3647
Epoch [6/20], Batch [1300/2382], Loss: 40.0324
Epoch [6/20], Batch [1400/2382], Loss: 83.4682
Epoch [6/20], Batch [1500/2382], Loss: 28.1905
Epoch [6/20], Batch [1600/2382], Loss: 54.3361
Epoch [6/20], Batch [1700/2382], Loss: 48.2084
Epoch [6/20], Batch [1800/2382], Loss: 23.8300
Epoch [6/20], Batch [1900/2382], Loss: 26.2829
Epoch [6/20], Batch [2000/2382], Loss: 38.7713
Epoch [6/20], Batch [2100/2382], Loss: 150.8731
Epoch [6/20], Batch [2200/2382], Loss: 58.4329
Epoch [6/20], Batch [2300/2382], Loss: 49.5752
Epoch [6/20] completed. Average Loss: 50.737937
Epoch [7/20], Batch [100/2382], Loss: 50.8002
Epoch [7/20], Batch [200/2382], Loss: 16.8922
Epoch [7/20], Batch [300/2382], Loss: 36.1644
Epoch [7/20], Batch [400/2382], Loss: 30.0266
Epoch [7/20], Batch [500/2382], Loss: 39.2016
Epoch [7/20], Batch [600/2382], Loss: 27.1806
Epoch [7/20], Batch [700/2382], Loss: 46.1249
Epoch [7/20], Batch [800/2382], Loss: 43.4221
Epoch [7/20], Batch [900/2382], Loss: 49.9242
Epoch [7/20], Batch [1000/2382], Loss: 43.3770
Epoch [7/20], Batch [1100/2382], Loss: 20.2412
Epoch [7/20], Batch [1200/2382], Loss: 137.6553
Epoch [7/20], Batch [1300/2382], Loss: 111.3945
Epoch [7/20], Batch [1400/2382], Loss: 66.4980
Epoch [7/20], Batch [1500/2382], Loss: 29.5111
Epoch [7/20], Batch [1600/2382], Loss: 39.2695
Epoch [7/20], Batch [1700/2382], Loss: 24.2365
Epoch [7/20], Batch [1800/2382], Loss: 25.2852
Epoch [7/20], Batch [1900/2382], Loss: 52.0916
Epoch [7/20], Batch [2000/2382], Loss: 38.8840
Epoch [7/20], Batch [2100/2382], Loss: 29.6086
Epoch [7/20], Batch [2200/2382], Loss: 36.6972
Epoch [7/20], Batch [2300/2382], Loss: 35.5856

Epoch [7/20] completed. Average Loss: 47.538108
Epoch [8/20], Batch [100/2382], Loss: 77.3128
Epoch [8/20], Batch [200/2382], Loss: 25.0031
Epoch [8/20], Batch [300/2382], Loss: 40.6967
Epoch [8/20], Batch [400/2382], Loss: 25.0690
Epoch [8/20], Batch [500/2382], Loss: 16.8186
Epoch [8/20], Batch [600/2382], Loss: 35.8342
Epoch [8/20], Batch [700/2382], Loss: 62.2433
Epoch [8/20], Batch [800/2382], Loss: 30.5434
Epoch [8/20], Batch [900/2382], Loss: 58.6734
Epoch [8/20], Batch [1000/2382], Loss: 50.5547
Epoch [8/20], Batch [1100/2382], Loss: 29.6182
Epoch [8/20], Batch [1200/2382], Loss: 91.3883
Epoch [8/20], Batch [1300/2382], Loss: 21.2038
Epoch [8/20], Batch [1400/2382], Loss: 36.6940
Epoch [8/20], Batch [1500/2382], Loss: 65.7082
Epoch [8/20], Batch [1600/2382], Loss: 30.9436
Epoch [8/20], Batch [1700/2382], Loss: 32.5596
Epoch [8/20], Batch [1800/2382], Loss: 29.2395
Epoch [8/20], Batch [1900/2382], Loss: 92.0902
Epoch [8/20], Batch [2000/2382], Loss: 64.9159
Epoch [8/20], Batch [2100/2382], Loss: 41.7610
Epoch [8/20], Batch [2200/2382], Loss: 49.4479
Epoch [8/20], Batch [2300/2382], Loss: 20.1273
Epoch [8/20] completed. Average Loss: 47.673368
Epoch [9/20], Batch [100/2382], Loss: 47.2927
Epoch [9/20], Batch [200/2382], Loss: 25.5471
Epoch [9/20], Batch [300/2382], Loss: 46.8817
Epoch [9/20], Batch [400/2382], Loss: 93.4236
Epoch [9/20], Batch [500/2382], Loss: 27.6539
Epoch [9/20], Batch [600/2382], Loss: 15.9992
Epoch [9/20], Batch [700/2382], Loss: 55.4565
Epoch [9/20], Batch [800/2382], Loss: 26.7437
Epoch [9/20], Batch [900/2382], Loss: 41.8624
Epoch [9/20], Batch [1000/2382], Loss: 36.0948
Epoch [9/20], Batch [1100/2382], Loss: 33.6910
Epoch [9/20], Batch [1200/2382], Loss: 69.1793
Epoch [9/20], Batch [1300/2382], Loss: 52.6031
Epoch [9/20], Batch [1400/2382], Loss: 23.5385
Epoch [9/20], Batch [1500/2382], Loss: 80.7792
Epoch [9/20], Batch [1600/2382], Loss: 86.0214
Epoch [9/20], Batch [1700/2382], Loss: 62.1871
Epoch [9/20], Batch [1800/2382], Loss: 74.5174
Epoch [9/20], Batch [1900/2382], Loss: 16.7576
Epoch [9/20], Batch [2000/2382], Loss: 34.4121
Epoch [9/20], Batch [2100/2382], Loss: 33.7641
Epoch [9/20], Batch [2200/2382], Loss: 40.2958
Epoch [9/20], Batch [2300/2382], Loss: 28.2047
Epoch [9/20] completed. Average Loss: 47.448785
Epoch [10/20], Batch [100/2382], Loss: 61.6747
Epoch [10/20], Batch [200/2382], Loss: 141.1509
Epoch [10/20], Batch [300/2382], Loss: 78.4881
Epoch [10/20], Batch [400/2382], Loss: 65.8156
Epoch [10/20], Batch [500/2382], Loss: 38.6104
Epoch [10/20], Batch [600/2382], Loss: 45.3959
Epoch [10/20], Batch [700/2382], Loss: 16.4087

Epoch [10/20], Batch [800/2382], Loss: 31.2418
Epoch [10/20], Batch [900/2382], Loss: 22.0856
Epoch [10/20], Batch [1000/2382], Loss: 18.2991
Epoch [10/20], Batch [1100/2382], Loss: 85.3029
Epoch [10/20], Batch [1200/2382], Loss: 23.1925
Epoch [10/20], Batch [1300/2382], Loss: 19.1224
Epoch [10/20], Batch [1400/2382], Loss: 54.5268
Epoch [10/20], Batch [1500/2382], Loss: 68.2064
Epoch [10/20], Batch [1600/2382], Loss: 30.0186
Epoch [10/20], Batch [1700/2382], Loss: 42.3155
Epoch [10/20], Batch [1800/2382], Loss: 34.5885
Epoch [10/20], Batch [1900/2382], Loss: 24.1779
Epoch [10/20], Batch [2000/2382], Loss: 48.9154
Epoch [10/20], Batch [2100/2382], Loss: 43.6658
Epoch [10/20], Batch [2200/2382], Loss: 63.3064
Epoch [10/20], Batch [2300/2382], Loss: 36.2314
Epoch [10/20] completed. Average Loss: 43.245615
Epoch [11/20], Batch [100/2382], Loss: 83.3542
Epoch [11/20], Batch [200/2382], Loss: 38.1985
Epoch [11/20], Batch [300/2382], Loss: 32.8120
Epoch [11/20], Batch [400/2382], Loss: 20.4415
Epoch [11/20], Batch [500/2382], Loss: 46.6782
Epoch [11/20], Batch [600/2382], Loss: 41.9908
Epoch [11/20], Batch [700/2382], Loss: 81.2655
Epoch [11/20], Batch [800/2382], Loss: 58.0560
Epoch [11/20], Batch [900/2382], Loss: 31.5679
Epoch [11/20], Batch [1000/2382], Loss: 26.8665
Epoch [11/20], Batch [1100/2382], Loss: 26.8192
Epoch [11/20], Batch [1200/2382], Loss: 23.1404
Epoch [11/20], Batch [1300/2382], Loss: 41.6148
Epoch [11/20], Batch [1400/2382], Loss: 37.6717
Epoch [11/20], Batch [1500/2382], Loss: 38.6834
Epoch [11/20], Batch [1600/2382], Loss: 34.2253
Epoch [11/20], Batch [1700/2382], Loss: 40.8889
Epoch [11/20], Batch [1800/2382], Loss: 37.7495
Epoch [11/20], Batch [1900/2382], Loss: 14.5100
Epoch [11/20], Batch [2000/2382], Loss: 44.5836
Epoch [11/20], Batch [2100/2382], Loss: 30.5191
Epoch [11/20], Batch [2200/2382], Loss: 32.2807
Epoch [11/20], Batch [2300/2382], Loss: 27.5323
Epoch [11/20] completed. Average Loss: 43.552233
Epoch [12/20], Batch [100/2382], Loss: 22.0684
Epoch [12/20], Batch [200/2382], Loss: 21.8477
Epoch [12/20], Batch [300/2382], Loss: 57.8601
Epoch [12/20], Batch [400/2382], Loss: 35.4256
Epoch [12/20], Batch [500/2382], Loss: 13.4175
Epoch [12/20], Batch [600/2382], Loss: 13.8490
Epoch [12/20], Batch [700/2382], Loss: 35.6541
Epoch [12/20], Batch [800/2382], Loss: 61.5857
Epoch [12/20], Batch [900/2382], Loss: 45.7486
Epoch [12/20], Batch [1000/2382], Loss: 40.9052
Epoch [12/20], Batch [1100/2382], Loss: 25.6926
Epoch [12/20], Batch [1200/2382], Loss: 59.9949
Epoch [12/20], Batch [1300/2382], Loss: 41.1745
Epoch [12/20], Batch [1400/2382], Loss: 40.9558
Epoch [12/20], Batch [1500/2382], Loss: 29.4240

Epoch [12/20], Batch [1600/2382], Loss: 15.9326
Epoch [12/20], Batch [1700/2382], Loss: 21.8536
Epoch [12/20], Batch [1800/2382], Loss: 37.4453
Epoch [12/20], Batch [1900/2382], Loss: 28.8793
Epoch [12/20], Batch [2000/2382], Loss: 40.8897
Epoch [12/20], Batch [2100/2382], Loss: 27.7635
Epoch [12/20], Batch [2200/2382], Loss: 30.4426
Epoch [12/20], Batch [2300/2382], Loss: 32.0890
Epoch [12/20] completed. Average Loss: 40.828368
Epoch [13/20], Batch [100/2382], Loss: 51.8220
Epoch [13/20], Batch [200/2382], Loss: 17.9606
Epoch [13/20], Batch [300/2382], Loss: 37.9719
Epoch [13/20], Batch [400/2382], Loss: 15.4211
Epoch [13/20], Batch [500/2382], Loss: 42.9535
Epoch [13/20], Batch [600/2382], Loss: 36.0476
Epoch [13/20], Batch [700/2382], Loss: 33.8507
Epoch [13/20], Batch [800/2382], Loss: 45.6859
Epoch [13/20], Batch [900/2382], Loss: 35.4022
Epoch [13/20], Batch [1000/2382], Loss: 27.4807
Epoch [13/20], Batch [1100/2382], Loss: 34.0720
Epoch [13/20], Batch [1200/2382], Loss: 53.8059
Epoch [13/20], Batch [1300/2382], Loss: 25.6120
Epoch [13/20], Batch [1400/2382], Loss: 108.2034
Epoch [13/20], Batch [1500/2382], Loss: 22.1666
Epoch [13/20], Batch [1600/2382], Loss: 15.9499
Epoch [13/20], Batch [1700/2382], Loss: 33.5086
Epoch [13/20], Batch [1800/2382], Loss: 48.6284
Epoch [13/20], Batch [1900/2382], Loss: 64.6265
Epoch [13/20], Batch [2000/2382], Loss: 21.9150
Epoch [13/20], Batch [2100/2382], Loss: 45.8790
Epoch [13/20], Batch [2200/2382], Loss: 27.8329
Epoch [13/20], Batch [2300/2382], Loss: 32.4951
Epoch [13/20] completed. Average Loss: 39.832092
Epoch [14/20], Batch [100/2382], Loss: 63.4249
Epoch [14/20], Batch [200/2382], Loss: 26.8944
Epoch [14/20], Batch [300/2382], Loss: 41.6234
Epoch [14/20], Batch [400/2382], Loss: 27.5059
Epoch [14/20], Batch [500/2382], Loss: 20.3220
Epoch [14/20], Batch [600/2382], Loss: 29.4292
Epoch [14/20], Batch [700/2382], Loss: 41.0549
Epoch [14/20], Batch [800/2382], Loss: 55.0486
Epoch [14/20], Batch [900/2382], Loss: 57.9074
Epoch [14/20], Batch [1000/2382], Loss: 36.9259
Epoch [14/20], Batch [1100/2382], Loss: 46.7794
Epoch [14/20], Batch [1200/2382], Loss: 30.8856
Epoch [14/20], Batch [1300/2382], Loss: 24.8290
Epoch [14/20], Batch [1400/2382], Loss: 42.4246
Epoch [14/20], Batch [1500/2382], Loss: 41.6469
Epoch [14/20], Batch [1600/2382], Loss: 16.8700
Epoch [14/20], Batch [1700/2382], Loss: 32.0389
Epoch [14/20], Batch [1800/2382], Loss: 29.4998
Epoch [14/20], Batch [1900/2382], Loss: 43.0103
Epoch [14/20], Batch [2000/2382], Loss: 61.7666
Epoch [14/20], Batch [2100/2382], Loss: 14.2496
Epoch [14/20], Batch [2200/2382], Loss: 10.4323
Epoch [14/20], Batch [2300/2382], Loss: 21.2056

Epoch [14/20] completed. Average Loss: 40.519201
Epoch [15/20], Batch [100/2382], Loss: 29.6543
Epoch [15/20], Batch [200/2382], Loss: 18.1684
Epoch [15/20], Batch [300/2382], Loss: 18.9829
Epoch [15/20], Batch [400/2382], Loss: 19.6263
Epoch [15/20], Batch [500/2382], Loss: 49.7813
Epoch [15/20], Batch [600/2382], Loss: 19.5974
Epoch [15/20], Batch [700/2382], Loss: 23.3304
Epoch [15/20], Batch [800/2382], Loss: 14.4081
Epoch [15/20], Batch [900/2382], Loss: 21.7141
Epoch [15/20], Batch [1000/2382], Loss: 29.5878
Epoch [15/20], Batch [1100/2382], Loss: 32.9587
Epoch [15/20], Batch [1200/2382], Loss: 54.7266
Epoch [15/20], Batch [1300/2382], Loss: 37.9992
Epoch [15/20], Batch [1400/2382], Loss: 47.5031
Epoch [15/20], Batch [1500/2382], Loss: 22.9864
Epoch [15/20], Batch [1600/2382], Loss: 67.6197
Epoch [15/20], Batch [1700/2382], Loss: 72.1596
Epoch [15/20], Batch [1800/2382], Loss: 50.7735
Epoch [15/20], Batch [1900/2382], Loss: 54.8976
Epoch [15/20], Batch [2000/2382], Loss: 33.6016
Epoch [15/20], Batch [2100/2382], Loss: 58.2207
Epoch [15/20], Batch [2200/2382], Loss: 98.2959
Epoch [15/20], Batch [2300/2382], Loss: 39.9781
Epoch [15/20] completed. Average Loss: 40.807792
Epoch [16/20], Batch [100/2382], Loss: 18.8545
Epoch [16/20], Batch [200/2382], Loss: 47.1668
Epoch [16/20], Batch [300/2382], Loss: 18.5786
Epoch [16/20], Batch [400/2382], Loss: 34.6831
Epoch [16/20], Batch [500/2382], Loss: 36.9414
Epoch [16/20], Batch [600/2382], Loss: 34.5807
Epoch [16/20], Batch [700/2382], Loss: 31.7025
Epoch [16/20], Batch [800/2382], Loss: 26.2077
Epoch [16/20], Batch [900/2382], Loss: 45.6888
Epoch [16/20], Batch [1000/2382], Loss: 24.2667
Epoch [16/20], Batch [1100/2382], Loss: 40.6175
Epoch [16/20], Batch [1200/2382], Loss: 26.4956
Epoch [16/20], Batch [1300/2382], Loss: 45.3782
Epoch [16/20], Batch [1400/2382], Loss: 41.3530
Epoch [16/20], Batch [1500/2382], Loss: 14.6758
Epoch [16/20], Batch [1600/2382], Loss: 19.2702
Epoch [16/20], Batch [1700/2382], Loss: 38.9579
Epoch [16/20], Batch [1800/2382], Loss: 38.5527
Epoch [16/20], Batch [1900/2382], Loss: 33.0619
Epoch [16/20], Batch [2000/2382], Loss: 32.9587
Epoch [16/20], Batch [2100/2382], Loss: 45.9415
Epoch [16/20], Batch [2200/2382], Loss: 31.8840
Epoch [16/20], Batch [2300/2382], Loss: 29.7435
Epoch [16/20] completed. Average Loss: 37.449351
Epoch [17/20], Batch [100/2382], Loss: 37.4704
Epoch [17/20], Batch [200/2382], Loss: 17.6400
Epoch [17/20], Batch [300/2382], Loss: 31.2018
Epoch [17/20], Batch [400/2382], Loss: 24.8856
Epoch [17/20], Batch [500/2382], Loss: 21.4031
Epoch [17/20], Batch [600/2382], Loss: 47.8610
Epoch [17/20], Batch [700/2382], Loss: 31.8361

Epoch [17/20], Batch [800/2382], Loss: 25.9860
Epoch [17/20], Batch [900/2382], Loss: 61.4884
Epoch [17/20], Batch [1000/2382], Loss: 31.0635
Epoch [17/20], Batch [1100/2382], Loss: 47.2380
Epoch [17/20], Batch [1200/2382], Loss: 60.3060
Epoch [17/20], Batch [1300/2382], Loss: 32.5154
Epoch [17/20], Batch [1400/2382], Loss: 59.1499
Epoch [17/20], Batch [1500/2382], Loss: 52.5863
Epoch [17/20], Batch [1600/2382], Loss: 29.1205
Epoch [17/20], Batch [1700/2382], Loss: 34.7915
Epoch [17/20], Batch [1800/2382], Loss: 37.9911
Epoch [17/20], Batch [1900/2382], Loss: 36.5494
Epoch [17/20], Batch [2000/2382], Loss: 20.6700
Epoch [17/20], Batch [2100/2382], Loss: 31.9321
Epoch [17/20], Batch [2200/2382], Loss: 20.2555
Epoch [17/20], Batch [2300/2382], Loss: 13.7654
Epoch [17/20] completed. Average Loss: 37.907465
Epoch [18/20], Batch [100/2382], Loss: 8.2036
Epoch [18/20], Batch [200/2382], Loss: 28.4803
Epoch [18/20], Batch [300/2382], Loss: 20.0832
Epoch [18/20], Batch [400/2382], Loss: 28.8797
Epoch [18/20], Batch [500/2382], Loss: 52.0992
Epoch [18/20], Batch [600/2382], Loss: 39.0461
Epoch [18/20], Batch [700/2382], Loss: 61.2967
Epoch [18/20], Batch [800/2382], Loss: 57.0158
Epoch [18/20], Batch [900/2382], Loss: 25.0352
Epoch [18/20], Batch [1000/2382], Loss: 41.2081
Epoch [18/20], Batch [1100/2382], Loss: 31.0301
Epoch [18/20], Batch [1200/2382], Loss: 67.8309
Epoch [18/20], Batch [1300/2382], Loss: 46.9518
Epoch [18/20], Batch [1400/2382], Loss: 23.2515
Epoch [18/20], Batch [1500/2382], Loss: 79.1818
Epoch [18/20], Batch [1600/2382], Loss: 24.9406
Epoch [18/20], Batch [1700/2382], Loss: 22.6952
Epoch [18/20], Batch [1800/2382], Loss: 34.3056
Epoch [18/20], Batch [1900/2382], Loss: 69.4856
Epoch [18/20], Batch [2000/2382], Loss: 58.9103
Epoch [18/20], Batch [2100/2382], Loss: 18.6803
Epoch [18/20], Batch [2200/2382], Loss: 31.4838
Epoch [18/20], Batch [2300/2382], Loss: 14.4825
Epoch [18/20] completed. Average Loss: 38.663291
Epoch [19/20], Batch [100/2382], Loss: 32.0461
Epoch [19/20], Batch [200/2382], Loss: 31.2719
Epoch [19/20], Batch [300/2382], Loss: 52.8485
Epoch [19/20], Batch [400/2382], Loss: 24.1825
Epoch [19/20], Batch [500/2382], Loss: 31.3094
Epoch [19/20], Batch [600/2382], Loss: 50.5585
Epoch [19/20], Batch [700/2382], Loss: 17.4972
Epoch [19/20], Batch [800/2382], Loss: 75.6007
Epoch [19/20], Batch [900/2382], Loss: 29.2315
Epoch [19/20], Batch [1000/2382], Loss: 30.9279
Epoch [19/20], Batch [1100/2382], Loss: 71.8355
Epoch [19/20], Batch [1200/2382], Loss: 37.5454
Epoch [19/20], Batch [1300/2382], Loss: 19.2064
Epoch [19/20], Batch [1400/2382], Loss: 43.2563
Epoch [19/20], Batch [1500/2382], Loss: 22.9402

```
Epoch [19/20], Batch [1600/2382], Loss: 55.0937
Epoch [19/20], Batch [1700/2382], Loss: 27.8765
Epoch [19/20], Batch [1800/2382], Loss: 44.8094
Epoch [19/20], Batch [1900/2382], Loss: 34.0338
Epoch [19/20], Batch [2000/2382], Loss: 51.3921
Epoch [19/20], Batch [2100/2382], Loss: 38.0873
Epoch [19/20], Batch [2200/2382], Loss: 10.7228
Epoch [19/20], Batch [2300/2382], Loss: 25.7346
Epoch [19/20] completed. Average Loss: 36.644302
Epoch [20/20], Batch [100/2382], Loss: 62.0757
Epoch [20/20], Batch [200/2382], Loss: 40.0564
Epoch [20/20], Batch [300/2382], Loss: 16.6166
Epoch [20/20], Batch [400/2382], Loss: 33.4950
Epoch [20/20], Batch [500/2382], Loss: 31.8744
Epoch [20/20], Batch [600/2382], Loss: 28.1411
Epoch [20/20], Batch [700/2382], Loss: 25.0359
Epoch [20/20], Batch [800/2382], Loss: 34.8540
Epoch [20/20], Batch [900/2382], Loss: 18.0105
Epoch [20/20], Batch [1000/2382], Loss: 17.1675
Epoch [20/20], Batch [1100/2382], Loss: 20.4954
Epoch [20/20], Batch [1200/2382], Loss: 37.5255
Epoch [20/20], Batch [1300/2382], Loss: 38.4590
Epoch [20/20], Batch [1400/2382], Loss: 25.8364
Epoch [20/20], Batch [1500/2382], Loss: 8.8687
Epoch [20/20], Batch [1600/2382], Loss: 26.7942
Epoch [20/20], Batch [1700/2382], Loss: 32.9466
Epoch [20/20], Batch [1800/2382], Loss: 43.0350
Epoch [20/20], Batch [1900/2382], Loss: 26.9013
Epoch [20/20], Batch [2000/2382], Loss: 18.5928
Epoch [20/20], Batch [2100/2382], Loss: 36.0176
Epoch [20/20], Batch [2200/2382], Loss: 59.8369
Epoch [20/20], Batch [2300/2382], Loss: 41.9724
Epoch [20/20] completed. Average Loss: 36.392093
Training finished.
```

Test model inference

```
In [18]: x_digits_test = []
         y_labels_test = [] # To store 1 or 2

         for i in range(len(x_test)):
             if y_test[i] == 1 or y_test[i] == 2:
                 x_digits_test.append(x_test[i])
                 y_labels_test.append(y_test[i]) # Store the label (1 or 2)

         print(f"Found {len(x_digits_test)} images of digits 1 and 2 in TEST.")
         # Convert labels to numpy array for easier handling later if needed
         y_labels_test = np.array(y_labels_test)
```

Found 2167 images of digits 1 and 2 in TEST.

```
In [19]: rotated_images_all_test = {} # Dictionary to store images by angle

         for angle in angles:
```

```

print(f"Rotating images by {angle} degrees...")
rotated_list_for_angle_test = []
if angle == 0:
    # No rotation needed for 0 degrees
    rotated_list_for_angle = [img for img in x_digits_test] # Use original images
else:
    for img in x_digits_test:
        # Apply rotation [cite: 13] - Make sure reshape=False if you want to keep original shape
        # If reshape=True, images might get larger, which could affect your VAE input requirements. Assuming reshape=False might be better
        rotated_img_test = rotate(img, angle, reshape=False, mode='nearest')
        rotated_list_for_angle_test.append(rotated_img_test)

    # Store the list of numpy arrays for this angle
    rotated_images_all_test[angle] = np.array(rotated_list_for_angle_test) # Convert list to array

print("Finished rotating images for all angles.")

```

```

Rotating images by 0 degrees...
Rotating images by 30 degrees...
Rotating images by 60 degrees...
Rotating images by 90 degrees...
Rotating images by 120 degrees...
Rotating images by 150 degrees...
Rotating images by 180 degrees...
Rotating images by 210 degrees...
Rotating images by 240 degrees...
Rotating images by 270 degrees...
Rotating images by 300 degrees...
Rotating images by 330 degrees...
Finished rotating images for all angles.

```

```
In [22]: images_np_test = rotated_images_all_test[angle]
```

```

In [23]: latent_vectors_all_test = {}

with torch.no_grad():
    for angle in angles:
        print(f"Encoding images for {angle} degrees...")
        images_np_test = rotated_images_all_test[angle]

        if images_np_test.shape[0] == 0:
            print(f"Skipping {angle} degrees because there are no images.")
            continue

        # Convert numpy array to PyTorch tensor, add channel dimension
        images_tensor_test = torch.tensor(images_np_test, dtype=torch.float32)

        mu_test, logvar_test = model.encode(images_tensor_test)

        latent_vectors_all_test[angle] = mu_test.cpu().numpy()

print("Finished encoding images for all angles.")

```

```

Encoding images for 0 degrees...
Skipping 0 degrees because there are no images.
Encoding images for 30 degrees...
Encoding images for 60 degrees...
Encoding images for 90 degrees...
Encoding images for 120 degrees...
Encoding images for 150 degrees...
Encoding images for 180 degrees...
Encoding images for 210 degrees...
Encoding images for 240 degrees...
Encoding images for 270 degrees...
Encoding images for 300 degrees...
Encoding images for 330 degrees...
Finished encoding images for all angles.

```

```

In [31]: X_mlp_test = []
         y_mlp_test = []

         available_angles = list(latent_vectors_all_test.keys()) # Use only angles that have images
         angles_list = sorted(available_angles) # Make sure angles are in order

         for i in range(len(angles_list)):
             current_angle = angles_list[i]
             next_angle = angles_list[(i + 1) % len(angles_list)]

             # Skip if either current or next angle is missing
             if current_angle not in latent_vectors_all_test or next_angle not in latent_vectors_all_test:
                 print(f"Skipping pair ({current_angle}, {next_angle}) due to missing images")
                 continue

             z_theta = latent_vectors_all_test[current_angle]
             z_theta_plus_30 = latent_vectors_all_test[next_angle]

             X_mlp_test.append(z_theta)
             y_mlp_test.append(z_theta_plus_30)

         # Concatenate to get full arrays
         x_mlp_test = np.concatenate(X_mlp_test, axis=0)
         y_mlp_test = np.concatenate(y_mlp_test, axis=0)

         print(f"MLP Input Shape: {x_mlp_test.shape}")
         print(f"MLP Target Shape: {y_mlp_test.shape}")

```

```

MLP Input Shape: (23837, 16)
MLP Target Shape: (23837, 16)

```

```

In [32]: X_test_tensor = torch.tensor(x_mlp_test, dtype=torch.float32)
         y_test_tensor = torch.tensor(y_mlp_test, dtype=torch.float32)

         criterion = nn.MSELoss()

         with torch.no_grad(): # No need to calculate gradients during testing
             predictions = mlp_model(X_test_tensor)
             test_loss = criterion(predictions, y_test_tensor)

         print(f"Test Loss (MSE): {test_loss.item():.6f}")

```

Test Loss (MSE): 11798167.000000

```
In [ ]: mlp_model.eval()

# Ensure you have the test data available (e.g., x_digits_test)
# and the dictionary of actual rotated test images (rotated_images_all_test)
# and the dictionary of actual encoded test latent vectors (latent_vectors_all_test)

# --- Choose Samples to Visualize ---
num_samples_to_show = 5
# Select some indices from your test set
sample_indices = np.random.choice(len(x_digits_test), num_samples_to_show, replace=True)

# --- Perform Visualization ---
fig, axes = plt.subplots(num_samples_to_show, 4, figsize=(12, 3 * num_samples_to_show))
if num_samples_to_show == 1: # Handle case for single sample plot
    axes = np.array([axes])

fig.suptitle("MLP Visual Test: Original -> MLP Prediction -> Actual Rotated")

with torch.no_grad(): # Ensure no gradients are calculated
    for i, idx in enumerate(sample_indices):
        # 1. Get Original Image and Target Rotated Image
        original_img_np = x_digits_test[idx]
        target_rotated_img_np = rotated_images_all_test[30][idx] # Get the actual rotated image

        # 2. Encode Original Image (0 degrees)
        img_tensor = torch.tensor(original_img_np, dtype=torch.float32).unsqueeze(0)
        # Normalize if you did during training/encoding
        # img_tensor = img_tensor / 255.0
        # Move to device if needed: img_tensor = img_tensor.to(device)
        z_0, _ = model.encode(img_tensor) # Get latent vector (mu)

        # 3. Use MLP to Predict z_30
        # Ensure z_0 is on the correct device if MLP is on GPU
        z_pred_30 = mlp_model(z_0)

        # Ensure your latent_vectors_all_test is structured correctly
        z_actual_30 = torch.tensor(latent_vectors_all_test[30][idx], dtype=torch.float32).unsqueeze(0)

        # 5. Decode latent vectors back to images
        # Ensure z vectors are on the correct device if VAE model is on GPU
        recon_original = model.decode(z_0).cpu().squeeze().numpy()
        recon_pred_30 = model.decode(z_pred_30).cpu().squeeze().numpy()
        recon_actual_30 = model.decode(z_actual_30).cpu().squeeze().numpy()

        # 6. Plotting
        # Original Input Image
        ax = axes[i, 0]
        ax.imshow(original_img_np, cmap='gray')
        ax.set_title(f"Sample {idx}\nOriginal (0°)")
        ax.axis('off')

        # Reconstructed MLP Prediction
```

```
ax = axes[i, 1]
ax.imshow(recon_pred_30, cmap='gray')
ax.set_title("MLP Predicted (30°)\n(Decoded z_pred_30)")
ax.axis('off')

# Reconstructed Actual Target (Optional but good comparison)
ax = axes[i, 2]
ax.imshow(recon_actual_30, cmap='gray')
ax.set_title("Actual Target (30°)\n(Decoded z_actual_30)")
ax.axis('off')

# Actual Rotated Image
ax = axes[i, 3]
ax.imshow(target_rotated_img_np, cmap='gray')
ax.set_title("Ground Truth (30°)")
ax.axis('off')

plt.tight_layout(rect=[0, 0.03, 1, 0.97]) # Adjust layout to prevent title c
plt.show()
```

MLP Visual Test: Original -> MLP Prediction -> Actual Rotated (Decoded)

